

Unidade 1

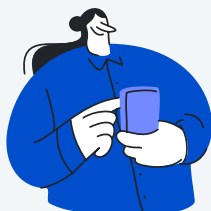
Introdução a Linguagem Python

Aula 1

A Linguagem Python

A linguagem Python

A linguagem Python



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a

sua formação profissional. Vamos assisti-la?

Bons estudos!

[Faça o download do arquivo](#)

Ponto de Partida

Ponto de Partida

Python é uma linguagem de programação de alto nível amplamente usada na indústria de tecnologia. Nesta aula, você começará a entender por que a

Python é tão popular e como pode ser utilizada em diversas aplicações.

Você conhecerá as ferramentas necessárias para dar início à programação em Python, o que inclui a instalação do Python em seu computador e a seleção de um ambiente de desenvolvimento adequado.

As variáveis são fundamentais na programação, pois permitem armazenar e manipular dados. Durante esta etapa de estudos, você aprenderá a criar variáveis e descobrirá os diferentes tipos de dados disponíveis em Python.

Como professor, preciso avaliar constantemente os estudantes. Sendo assim, quero automatizar a média de notas dos alunos. É possível fazer isso utilizando Python?

Vamos Começar!

Vamos Começar!

Introdução à linguagem Python

```
In [1]:  
print("hello world!")  
hello world!
```

Há uma lenda entre programadores segundo a qual se você não imprimir o “hello world” quando começar a aprender uma linguagem, não conseguirá assimilar nada sobre ela (Ciência da Computação, 2015). Para não correr tal risco,

essa foi a primeira linha de comando apresentada a você dentre as muitas que aprenderá nesta disciplina.

Python é uma linguagem de programação versátil e fácil de aprender. Foi criada por Guido van Rossum e lançada em 1991. Guido é o principal autor da Python, embora haja muitas contribuições de outros pesquisadores (Python v3.0.1 Documentation, [s. d.]). Desde então, tornou-se uma das linguagens mais populares do mundo por causa de sua legibilidade e sintaxe simples. Você pode se perguntar: “por que escolher Python?”. A resposta é clara: Python é usado em várias áreas, incluindo desenvolvimento web, automação, aprendizado de máquina e análise de dados.

De acordo com o guia de desenvolvimento para iniciantes Python (Python Wiki, 2022), trata-se de uma linguagem de programação orientada a objetos, clara e poderosa, comparável a Perl, Ruby, Scheme ou Java.

Python tem se mostrado uma linguagem muito eficiente e vem sendo amplamente adotada por profissionais na área de dados (Agarwal, 2023), destacando-se por sua sintaxe. Uma das principais filosofias de Guido van Rossum, o criador da linguagem, é que o código deve ser facilmente legível, uma vez que é lido com mais frequência do que é escrito. Isso é formalizado no PEP 8, o Guia de Estilo para Código

Python, que estabelece as diretrizes para a formatação, organização e estruturação do código (Rossum; Warsaw; Coghlan, 2023). Seguir essas diretrizes resulta em um código que é considerado “pythonic” – ou seja, que adere aos princípios descritos no PEP 8. Essas regras abrangem elementos como a maneira com que o código é formatado, o modo pelo qual as funções são definidas e organizadas, a forma de aplicação da indentação e outros aspectos relacionados à sintaxe do código Python.

Já conseguiu entender por que o Python é tão usado e cada vez mais aplicado a todo tipo de programação? O próximo passo é saber como obtemos o Python, e é isso o que estudaremos a seguir!

Ferramentas e interpretadores

Agora que temos um entendimento básico sobre essa linguagem essencial e sabemos que existe um conjunto de regras definido pelo PEP 8 para escrever o código Python de maneira consistente, a próxima pergunta que naturalmente surge é: onde exatamente escrevemos esses códigos em Python e de que forma visualizamos os resultados? A implementação de códigos em Python pode ser realizada em uma variedade de ambientes, seja no seu próprio computador ou em ambientes baseados na nuvem. No

entanto, independentemente da escolha do ambiente, um elemento essencial nesse contexto é o uso de um interpretador Python para executar seus códigos.

O passo a passo para a instalação do interpretador Python está disponível no site oficial dessa linguagem, tanto para Windows (Python Brasil, 2019) , quanto para outros sistemas operacionais (Python Brasil, 2023; [Python Brasil, 2016](#)).

Após a instalação, podemos utilizar o prompt de comando para fazer alguns testes e até mesmo programar com Python por lá. Porém aconselho utilizar uma Integrated Development Environment (IDE), isto é, um Ambiente de Desenvolvimento Integrado. Existem várias IDEs que podem ser utilizadas e que são ótimas, como o PyCharm (JetBrains, 2010) e o Visual Studio Code (VSCode) (Microsoft, [s. d.]).

Outra ferramenta que se destaca nesse cenário é o Python Anaconda (Anaconda, [s. d.]), que possui diversos recursos Python, sendo composta por bibliotecas e IDEs. O diferencial é o Jupyter Notebook, um ambiente de computação iterativa que permite a criação de documentos de notebook que incluem código ativo, gráficos, textos narrativos, etc. Outra vantagem do Jupyter Notebook é a capacidade de funcionar em um navegador de internet. No endereço [jupyter](#), você pode experimentar a ferramenta sem precisar de instalação.

A ferramenta que indico como meio de trabalho para esta disciplina é o Google Colab, pois essa plataforma possibilita que qualquer pessoa escreva e execute código Python a partir do navegador. O Colab é um servidor de Notebook Jupyter hospedado que não requer configuração para ser utilizado. Como é baseado no projeto de código aberto, permite que você use e compartilhe os Notebooks Jupyter com outros usuários sem precisar baixar, instalar ou executar nada.

Vamos nessa! Para usar o Colab, acesse: [colab](https://colab.research.google.com/).

Fazendo uma pequena retrospectiva, é possível afirmar que já temos um entendimento básico do Python e de onde programar. Agora vamos praticar!

Siga em Frente...

Siga em Frente...

Variáveis e tipos de dados

O fluxo de um algoritmo é a entrada, o processamento e a saída. Note que, para que o processamento ocorra, é necessário armazenar os valores da entrada, por exemplo. Assim surge o conceito de variável, que nada mais é do que um espaço alocado na memória RAM.

O interpretador Python consegue estabelecer o tipo de dado da variável observando seu valor. Confira alguns exemplos:

```
x = 10
nome = 'aluno'
nota = 8.75
fez_inscricao = True
```

Observe que `x` é um número inteiro, `nome` é uma string, `nota` é um número decimal e `fez_inscricao` é um booleano. Será que Python é capaz de definir o tipo de variável somente com base no valor fornecido em cada variável?

Utilizaremos a função `print()` e `type()`.

```
print(type(x))
print(type(nome))
print(type(nota))
```

```
print(type(fez_inscricao))
```

```
<class 'int'>
```

```
<class 'str'>
```

```
<class 'float'>
```

```
<class 'bool'>
```

Como esperado, o Python acertou todos os tipos de variáveis. Note, também, que em Python tudo é objeto. Sendo assim, os tipos de dados aparecem com a palavra “class”, que é uma classe.

Já conseguimos criar uma variável. Que tal melhorar o famoso “hello world”? Para isso, usaremos a função `input()`, que faz a leitura de um valor digitado.

```
nome = input()
```

```
print(nome)
```

```
Digite um nome: Estudante Querido
```

```
Estudante Querido
```

Ao executar o comando `input`, surge o campo para digitar o que será capturado – no nosso caso, “Estudante Querido”. Logo após, a função `print()` mostra a variável “nome”. Vamos melhorar nosso “hello world”!

Existem muitas formas de imprimir textos e variáveis em Python. Usaremos formatadores de caracteres (igual em C) e a f-string.

```
#formatadores de caracteres
print("Olá, %s, bem-vindo à disciplina de programação. Parabéns pelo
seu primeiro hello world" % (nome))
Olá, Estudante Querido, bem-vindo à disciplina de programação.
Parabéns pelo seu primeiro hello world
#F-string
print(f{nome}, bem-vindo à disciplina de programação. Parabéns pelo
seu primeiro hello world")
Olá, Estudante Querido, bem-vindo à disciplina de programação.
Parabéns pelo seu primeiro hello world
```

De acordo com o PEP 498 (Smith, [s. d.]), a melhor opção é utilizar “f-string”. Sendo assim, usaremos esse recurso em nossas aulas.

Esta etapa de aprendizagem consistiu em uma introdução básica do Python, suas respectivas ferramentas e alguns exemplos de uso. O objetivo desta aula era mostrar como surgiu a linguagem Python, onde conseguimos usá-la e como esse processo deve ser feito. Para concluir, a intenção final era deixar você curioso para aprender mais. Espero que isso tenha acontecido!

Vamos Exercitar?

Vamos Exercitar?

Você se lembra da minha pergunta inicial? É possível utilizar Python para automatizar as médias dos meus alunos? Depois de tudo o que vimos nesta aula, a resposta é sim!

```
Nota_1 = int(input())
Nota_2 = int(input())
Nota_3 = int(input())
Nota_4 = int(input())
#observe que utilizamos a função int(), pois, sem ela, o Python
entenderia que as notas seriam String

#condição para a aprovação do aluno.
if media >= 6:
    situacao =
else:
    situacao =

#dadas as notas, mostramos a média final e a situação do aluno.
print(f{media})
print(f{situacao})
```

Repare que utilizei duas funções condicionais:

- “if ≥ 6 ” (se a nota for maior ou igual a 6): aluno aprovado.
- “else” (se a nota for menor do que 6): aluno reprovado.

Essas funções são essenciais na programação. Estudaremos mais situações nas quais elas são aplicadas no decorrer desta disciplina.

Gostou dessa solução? Espero que sim! Agora, quero que você fique ainda mais curioso para aprender Python!

Saiba mais

Saiba mais

1. Uma leitura interessante para quem está começando a programar em Python é a do livro *Começando a programar em Python para leigos*, cujo link de acesso está disponível a seguir.

MUELLER, J. P. [Começando a programar em Python para leigos](#). Rio de Janeiro: Alta Books, 2020. E-book.

2. Outra dica para estudo e aprofundamento sobre esse tema é o livro *Use a cabeça! Python*.

BARRY, P. [Use a Cabeça! Python](#). 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

3. Para a aplicação e o conhecimento sobre o uso da linguagem Python no mercado financeiro, recomendo a leitura do livro *Python e mercado financeiro: programação para estudantes, investidores e analistas*.

CAETANO, M. A. L. [Python e mercado financeiro: programação para estudantes, investidores e analistas](#). São Paulo: Blucher, 2021. E-book.

4. Por fim, também encorajo você a navegar pelos seguintes sites: [python](#), [jupyter](#) e [colab](#).

Referências

Referências

A ORIGEM do “Hello World”. **Ciência da Computação**, 2015.

Disponível em:

<https://cienciacomputacao.com.br/curiosidade/a-origem-do-hello-world/>. Acesso em: 12 out. 2023.

AGARWAL, A. From C to Python, and everything... **LinkedIn**, 2023. Disponível em:

<https://www.linkedin.com/feed/update/urn:li:activity:7056881167563702272/>. Acesso em: 12 out. 2023.

ANACONDA. **Página inicial**, [s. d.]. Disponível em:

<https://www.anaconda.com/>. Acesso em: 12 out. 2023.

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

BEGINNER’S Guide. **Python Wiki**, 22 nov. 2022. Disponível em:

<https://wiki.python.org/moin/BeginnersGuide/Overview>.

Acesso em: 12 out. 2023.

CAETANO, M. A. L. **Python e mercado financeiro**:

programação para estudantes, investidores e analistas. São Paulo: Blucher, 2021. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555062410>. Acesso em: 12 out. 2023.

GOOGLE COLAB. **Página inicial**, [s. d.]. Disponível em: <https://colab.research.google.com/>. Acesso em: 12 out. 2023.

HISTORY and license. **Python v3.0.1 Documentation**, [s. d.]. Disponível em: <https://docs.python.org/3.0/license.html>. Acesso em: 12 out. 2023.

INSTALANDO o Python 3 no Linux. **Python Brasil**, 27 jun. 2023. Disponível em: <https://python.org.br/instalacao-linux/>. Acesso em: 12 out. 2023.

INSTALANDO o Python 3 no Mac OS X. **Python Brasil**, 1 nov. 2016. Disponível em: <https://python.org.br/instalacao-mac/>. Acesso em: 12 out. 2023.

INSTALANDO o Python 3 no Windows. **Python Brasil**, 30 nov. 2019. Disponível em: <https://python.org.br/instalacao-windows/>. Acesso em: 12 out. 2023.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

PYCHARM. **JetBrains**, 5 fev. 2010. Disponível em: <https://www.jetbrains.com/pycharm/>. Acesso em: 12 out. 2023.

ROSSUM, G. V.; WARSAW, B.; COGHLAN, A. PEP 8 – Style Guide for Python Code. **Python Enhancement Proposals**, 9 dez. 2023. Disponível em: <https://peps.python.org/pep-0008/#introduction>. Acesso em: 12 fev. 2023.

SMITH, E. V. PEP 498 – Literal String Interpolation. **Python Enhancement Proposals**, [s. d.]. Disponível em: <https://peps.python.org/pep-0498/>. Acesso em: 12 out. 2023.

TRY JUPYTER. **Jupyter**, [s. d.]. Disponível em: <https://jupyter.org/try>. Acesso em: 12 out. 2023.

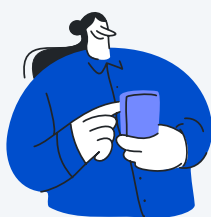
VISUAL Studio. **Microsoft**, [s. d.]. Disponível em: <https://visualstudio.microsoft.com/pt-br/>. Acesso em: 12 out. 2023.

Aula 2

Estruturas Condicionais em Python

Estruturas condicionais em Python

Estruturas condicionais em Python



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

Bons estudos!

[Faça o download do arquivo](#)

Ponto de Partida

Ponto de Partida

Dando continuidade ao nosso aprendizado sobre Python, nesta aula vamos conhecer três conceitos fundamentais para a implementação de algoritmos.

O primeiro deles consiste em uma ferramenta de comparação: os operadores relacionais. Eles nos permitem avaliar relações entre valores, respondendo a perguntas como: “é maior que?”, “é igual a?” ou “é diferente de?”.

O segundo conceito é o de estruturas lógicas, que são como peças de quebra-cabeça responsáveis por unir condições para criar critérios mais complexos. São elas que nos possibilitam tomar decisões mais elaboradas, combinando várias comparações.

Já com o terceiro conceito, de estruturas condicionais, dizemos ao programa o que fazer com base na seguinte condição: “se isso for verdade, aja assim; caso contrário, faça aquilo”.

A partir desse conhecimento, faremos uma aplicação dos conceitos mencionados por meio de um estudo de caso. Suponha que você trabalhe em uma empresa que cuida de cinemas. Em uma reunião com a diretoria, foi solicitada a implementação de um sistema de autoatendimento. Como se trata de algo novo na rede, você será responsável por elaborar um protótipo simples. A diretoria quer que o projeto a ser desenvolvido seja baseado na idade dos clientes e contenha a informação de disponibilidade de ingressos.

Vamos nessa?

Vamos Começar!

Vamos Começar!

Operadores relacionais

Na programação, a criação de algoritmos para resolver problemas envolve a capacidade de tomar decisões. Tais decisões são guiadas por uma técnica chamada “estrutura condicional” (Manzano; Oliveira, 2019). Pode-se entendê-la como a escolha de um caminho em uma cidade. Da mesma forma que ao dirigir na cidade você precisa decidir em quais ruas seguir para chegar ao seu destino, em um programa é necessário definir qual parte do código será executada em um determinado momento.

Nesse contexto, temos, na programação, os operadores relacionais (Quadro 1), que são usados para fazer comparações.

Operação	Significado
<	Estritamente menor que
<=	Menor ou igual que
>	Estritamente maior que
>=	Maior ou igual que
==	Igual
!=	Diferente

Operação	Significado
is	Identidade do objeto
is not	Negação da identidade do objeto

Quadro 1 | Operadores relacionais. Fonte: adaptado de Python 3.12.2 Documentation ([s. d.]).

Lembre-se de que já utilizamos um operador desse tipo na primeira aula quando estabelecemos a condição de que a média fosse maior ou igual a 6 para que o aluno recebesse a aprovação. A partir de agora, utilizaremos cada vez mais esses operadores para criar códigos mais robustos.

Estruturas lógicas

Além dos operadores relacionais, que comparam valores, também utilizamos operadores booleanos para construir decisões mais complexas em programação. Os operadores booleanos ajudam a combinar diferentes condições e a orientar o fluxo do programa de acordo com a lógica desejada.

- Operador “E” (and)
O operador “E” (and) permite a realização da operação lógica “E”. Isso significa que, ao usar a expressão (a and b), o resultado será “Verdadeiro” somente quando ambos os argumentos, “a” e “b”, forem verdadeiros. Caso contrário, o resultado será “Falso”.
- Operador “OU” (or)
O operador “OU” (or) realiza a operação lógica “OU”. Ao utilizar a expressão (a or b), o resultado será “Verdadeiro” se pelo menos um dos argumentos, “a” ou “b”, for verdadeiro. A expressão será “Falsa” apenas quando ambos os argumentos forem falsos.
- Operador “NÃO” (not)
O operador “NÃO” (not) é responsável por inverter o valor do argumento. Ao aplicarmos a expressão (not a), ela transformará “Verdadeiro” em “Falso”, e vice-versa. Ou seja, se o argumento for verdadeiro, a operação o tornará falso, e, se for falso, a operação o tornará verdadeiro.

Esses operadores booleanos são essenciais para a criação de estruturas de decisão mais sofisticadas, pois possibilitam que os programas lidem com uma variedade de situações e critérios lógicos. São usados para controlar o fluxo de execução com base em condições complexas e, assim, tornam viável a elaboração de programas que tomam decisões de acordo com critérios específicos.

Siga em Frente...

Siga em Frente...

Estruturas condicionais if, else e elif

No dia a dia, temos muitas escolhas a fazer, regras a seguir. Pense em você mesmo dirigindo. Se o semáforo está verde, você pode seguir; caso contrário, deve parar, pois está vermelho (desconsideraremos o amarelo por ora).

Esse contexto se relaciona com a estrutura if... else da seguinte maneira:

- A condição “se o semáforo estiver verde” é satisfeita (verdadeira), então você segue em frente (if).
- A condição “se o semáforo estiver vermelho” não é satisfeita (falsa), então você para (else).

Nesse exemplo, assim como acontece na programação, se uma condição é atendida, o fluxo de execução segue um caminho (verdadeiro); caso contrário, segue outro caminho

(falso). Os comandos `if` e `else` são como bifurcações em uma estrada, orientando o fluxo do programa com base nas condições estabelecidas.

O comando `elif`, em Python, é uma abreviação de “else if”, sendo usado em estruturas condicionais para avaliar múltiplas condições em sequência. É frequentemente utilizado após um bloco `if` e antes de um bloco `else`. A ideia por trás do `elif` é permitir que você verifique várias condições em ordem e, quando uma delas for verdadeira, o bloco de código associado a essa condição será executado.

Confira, a seguir, uma explicação mais detalhada sobre o `elif`:

1. **Avaliação em sequência:** quando um bloco `if` é usado, a condição é avaliada. Se a condição for verdadeira, o bloco de código dentro desse `if` é executado. No entanto, em muitos cenários, você deseja avaliar uma série de condições em sequência, e o `elif` permite esse processo.
2. **Verificação múltipla:** após o bloco `if`, você pode usar um ou mais blocos `elif`, cada um com sua própria condição. O Python avalia essas condições em ordem, do topo para baixo. Assim que uma condição for verdadeira, o bloco de código associado a essa condição será executado e as condições subsequentes serão ignoradas.

3. **Flexibilidade:** o `elif` é flexível porque permite que você trate de múltiplos casos sem precisar aninhar uma série de blocos `if`. Isso torna o código mais legível e eficiente.

Para resumir o que analisamos até agora, imagine que estejamos construindo uma máquina de venda automática de refrigerantes.

Os operadores relacionais são como os sensores que verificam se você inseriu a moeda correta ou não. Eles nos ajudam a determinar se a condição foi atendida (por exemplo: “a moeda é maior ou igual a R\$1,00?”).

As estruturas lógicas podem ser comparadas às engrenagens dessa máquina. Elas nos permitem combinar vários sensores para fazer verificações complexas (por exemplo: “se a moeda é maior ou igual a R\$1,00 e o refrigerante está disponível, então a máquina entrega um refrigerante”).

As estruturas condicionais são como as instruções para a máquina, pois nos permitem dizer o que a máquina deve fazer com base nas verificações. Se a máquina detectar que a moeda é suficiente (usando operadores relacionais e estruturas lógicas), ela seguirá as instruções para entregar o refrigerante. Caso contrário, talvez exiba uma mensagem de erro.

Então, assim como um engenheiro monta uma máquina de venda automática usando diferentes peças e sensores para tomar decisões, nós montamos nossos programas utilizando operadores relacionais, estruturas lógicas e estruturas condicionais para controlar o fluxo e a lógica do código.

Sendo assim, vamos verificar o exemplo de código a seguir. Fique à vontade para pensar, criar e executar códigos utilizando as regras apresentadas nesta aula.

```
idade = 25

if idade < 18:
    print("Menor de idade")
elif idade >= 18 and idade < 65:
    print("Adulto")
else:
    print("Idoso")
```

No exemplo anterior, usamos os três tipos de ferramentas que conhecemos durante esta etapa de aprendizagem. Repare na estrutura do if, elif e else. Ao final, definimos o que será feito com ":". Além disso, utilizamos operadores relacionais e o "and" para combinar duas condições.

Nesta aula, aprendemos sobre três tipos de ferramentas essenciais para criar algoritmos em Python. Com base nesse

novo conhecimento, você já se tornará capaz de produzir algoritmos mais elaborados. A prática é vital para aprender cada vez mais, então espero que você pratique bastante!

Vamos Exercitar?

Vamos Exercitar?

Agora, vamos colocar em prática um algoritmo de recomendação para um cinema, baseando-se na idade do cliente. Suponha que tenhamos três filmes por semana, cada um com uma faixa etária específica. O primeiro é direcionado para menores de 12 anos; o segundo, para maiores ou iguais a 12 anos e menores de 18; por fim, o terceiro filme é recomendado para maiores ou iguais a 18 anos. Outro ponto solicitado pela diretoria do cinema diz respeito à disponibilidade de ingressos. Vamos para o código!

```
# Bem-vindo à Máquina de Venda Automática de Ingressos de Cinema!
```

```
# Solicita a idade do cliente
```

```
idade = int(input("Por favor, digite sua idade: "))
```

```
# Verifica a idade para sugestão de filmes
if idade < 12:
    print("Recomendamos o filme infantil FILME 1.")
elif 12 <= idade < 18:
    print("Recomendamos o filme adolescente FILME 2.")
else:
    print("Recomendamos o emocionante FILME 3.")

# Verifica a disponibilidade de ingressos
quantidade_ingressos = 10 # Suponha que haja 10 ingressos
disponíveis
if quantidade_ingressos > 0:
    print("Ingressos estão disponíveis. Divirta-se no cinema!")
else:
    print("Desculpe, todos os ingressos estão esgotados para hoje.")
```

Rode esse código no seu Google Colab e faça testes, substitua a disponibilidade de ingresso, insira idades diferentes... Enfim, “brinque” com ele.

Gostou dessa solução? Espero que sim! Esta é a nossa segunda aula e já construímos um modelo de recomendação. Vamos seguir cada vez mais fundo nesse mundo de Python!

Saiba mais

Saiba mais

1. Para descobrir mais detalhes sobre sistemas de recomendação, é interessante que você leia o artigo [*Arquitetura de sistemas de recomendação para apoio ao vendedor no uso de sistemas de força de vendas em empresa com grande portfólio de produtos*](#), que mostra algumas aplicações e explica como funciona a utilização de tais modelos. Para acessar o conteúdo sugerido, clique no link disponível a seguir.
2. Uma leitura interessante para quem está começando a programar em Python é a do livro [*Começando a programar em Python para leigos*](#).
3. Outra dica para estudo e aprofundamento sobre esse tema é o livro [*Use a cabeça! Python*](#).

Referências

Referências

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

BUILT-IN Types. **Python 3.12.2 Documentation**, [s. d.].

Disponível em:

<https://docs.python.org/3/library/stdtypes.html>. Acesso em: 14 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

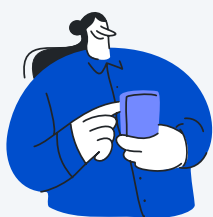
OHASHI, F. K. *et al.* Arquitetura de sistemas de recomendação para apoio ao vendedor no uso de sistemas de força de vendas em empresa com grande portfólio de produtos. **Revista Ibérica de Sistemas e Tecnologias de Informação**, Lousada, n. 42, p. 46-61, jun. 2021. Disponível em: <https://scielo.pt/pdf/rist/n42/1646-9895-rist-42-46.pdf>. Acesso em: 14 out. 2023.

Aula 3

Estruturas de Repetição em Python

Estruturas de repetição em Python

Estruturas de repetição em Python



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

Bons estudos!

[Faça o download do arquivo](#)

Ponto de Partida

Ponto de Partida

Avançando ainda mais em nossos estudos sobre Python, vamos conhecer, nesta aula, estruturas de repetição e de controle que são essenciais para a implementação de algoritmos.

Na intenção de simplificar tarefas repetitivas, você estudará o **“for”**, uma estrutura de repetição que permite percorrer uma sequência de elementos, como uma lista, e executar ações para cada item.

Já para controlar as repetições ou ter uma condição atendida, o **“while”** é a estrutura ideal. Nesta etapa de aprendizagem, você investigará o uso dessa ferramenta e entenderá por que sua implementação é essencial em certos algoritmos.

Para finalizar, você conhecerá as funções “**range**”, “**break**” e “**continue**”. Tais ferramentas também são usadas para controle de repetição. Ao final desta aula, você será capaz de definir a melhor opção a ser adotada para um determinado caso.

Suponha, agora, que você, ainda trabalhando no cinema do estudo de caso da aula passada, precise verificar a aceitação de cinco filmes exibidos no mês.

Para tanto, será necessário criar um algoritmo que seja capaz de classificar os filmes de 1 a 5 estrelas. Como posso utilizar estruturas de repetição e controle de repetição para elaborar esse algoritmo?

Vamos Começar!

Vamos Começar!

Estrutura de repetição for

A estrutura de repetição **for**, em Python, é uma poderosa ferramenta que nos permite realizar ações repetitivas de

maneira controlada (Manzano; Oliveira, 2019). Em um loop **for**, especificamos um conjunto de elementos (por exemplo, uma lista ou uma sequência) e, em seguida, o código é executado para cada elemento desse conjunto. Tal estrutura é especialmente útil quando sabemos previamente quantas vezes queremos repetir uma ação ou quando temos uma coleção de itens a serem processados.

Confira, a seguir, o exemplo simples de um loop **for** que itera por uma lista de números e imprime cada número:

```
numeros = [1, 2, 3, 4, 5]

for numero in numeros:
    print(numero)
```

Nesse exemplo, a variável 'numero' assume o valor de cada elemento da lista 'numeros' em sequência, e o bloco de código dentro do loop é executado para

cada valor. Isso resultará na impressão dos números de 1 a 5.

```
1
2
3
```

```
4  
5
```

O loop **for** é uma ferramenta eficaz para automatizar tarefas repetitivas e processar coleções de dados de modo eficiente.

Estrutura de repetição **while**

O comando **while** é uma ferramenta muito importante na programação Python, sendo usado para criar estruturas de repetição quando o número de repetições não é conhecido antecipadamente. Ele permite que um bloco de código seja executado repetidamente enquanto uma condição especificada for verdadeira. Isso torna o **while** ideal para situações em que a execução deve continuar até que uma condição específica seja atendida.

Acompanhe, a seguir, um exemplo simples de uso do **while** para verificar se um número inserido pelo usuário é par ou ímpar e encerrar o programa quando o número zero for inserido:

```
1 numero = int(input())  
2  
3 while numero != 0:  
4     if numero % 2 == 0:
```

```
5     print()
6 else:
7     print()
8     numero = int(input())
```

Nesse caso, o bloco de código dentro do **while** é executado repetidamente enquanto a condição **numero != 0** for verdadeira. Isso permite que o programa solicite ao usuário números repetidamente até que o número zero seja inserido, encerrando o programa. O **while** é uma ferramenta valiosa para lidar com situações em que a iteração é necessária, mas o número de repetições não é conhecido com antecedência.

Siga em Frente...

Siga em Frente...

Controle de repetição: range, break e continue

A função **range()**, em Python, é uma ferramenta útil para criar sequências numéricas que podem ser usadas em

estruturas de repetição, como o comando **for**. Ela oferece flexibilidade ao especificar os limites e o incremento da sequência.

Método 1 – Repetição por quantidade

No primeiro método, você pode passar um único argumento para **range()**, que representa a quantidade de vezes em que o loop deve ser repetido. Por exemplo, **range(5)** cria uma sequência que se inicia em 0 e vai até 4, realizando 5 iterações.

```
for x in range(5):  
    print(x)  
#a saída será:  
0  
1  
2  
3  
4
```

Método 2 – Limites inicial e superior

No segundo método, você pode fornecer dois argumentos para **range()**. O primeiro argumento representa o início das repetições e o segundo, o limite superior (não incluso) do valor da variável de controle. Por exemplo, **range(2, 7)** cria

uma sequência que se inicia em 2 e vai até 6, realizando 5 iterações.

```
for y in range(2, 7):  
    print(y)  
#a saída será:  
2  
3  
4  
5  
6
```

Método 3 – Com incremento

No terceiro método, você pode passar três argumentos para **range()**. O primeiro argumento é o início das repetições, o segundo é o limite superior (não incluso) e o terceiro argumento representa o incremento entre cada iteração. Por exemplo, **range(1, 11, 2)** cria uma sequência que começa em 1, vai até 10 e incrementa de 2 em 2, resultando nas iterações de 1, 3, 5, 7 e 9.

```
for z in range(1, 11, 2):  
    print(z)  
#a saída será:  
1  
3  
5
```

7
9

Além de controlar iterações com base na sequência de números ou condições específicas, Python oferece dois comandos eficientes para influenciar o fluxo de execução em estruturas de repetição: **“break”** e **“continue”**. Vamos entender como esses comandos funcionam e analisar alguns exemplos práticos.

O comando **“break”** é usado para interromper a execução de uma estrutura de repetição quando uma determinada condição é atendida. Essencialmente, esse comando permite sair do loop antes que ele seja concluído. Isso é útil quando você deseja encerrar um loop prematuramente com base em algum critério.

Suponha que desejemos encontrar o primeiro número par em uma sequência e interromper a iteração assim que o acharmos:

```
for numero in range(1, 11):  
    if numero % 2 == 0:  
        print(, numero)  
        break  
#a saída será:  
O primeiro número par encontrado é: 2
```

Nesse exemplo, o loop “**for**” itera de 1 a 10, mas, assim que encontra o primeiro número par (2), o comando “**break**” é acionado. Desse modo, interrompe-se a execução do loop.

O comando “**continue**” é usado para pular a iteração atual em uma estrutura de repetição e continuar com a próxima iteração. Isso é vantajoso quando você deseja ignorar uma iteração com base em uma condição, mas quer continuar com o restante do loop.

Vamos considerar um loop que imprime todos os números de 1 a 10, exceto o número 5:

```
for numero in range(1, 11):  
    if numero == 5:  
        continue  
    print(numero)  
#a saída será:  
1  
2  
3
```

```
4  
6  
7  
8  
9  
10
```

Nesse caso, quando o número é igual a 5, o comando **“continue”** é acionado, fazendo com que a iteração atual seja abandonada. A execução continua com o próximo número.

Em resumo, o **“break”** e o **“continue”** são comandos úteis para controlar o fluxo em estruturas de repetição, permitindo interromper loops antecipadamente com **“break”** e pular iterações específicas com **“continue”**, com base em condições específicas. Essas ferramentas adicionam flexibilidade ao controle de repetições em Python.

Vamos Exercitar?

Vamos Exercitar?

Vamos pensar na solução do problema apresentado no início desta aula. Precisamos criar um programa que seja capaz de

percorrer todos os filmes (Filme 1, Filme 2, Filme 3, Filme 4 e Filme 5) e de atribuir a cada um deles uma nota de 1 a 5. Repare que é importante sempre disponibilizar uma forma de a pessoa encerrar o programa, caso queira.

```
filmes = [ , , , ]

print()
print()
print()

for filme in filmes:
    while True:
        classificacao = input(f'{filme}' de 1 a 5? (ou 0 para parar): ")
        if classificacao == '0':
            print(f'{filme}' interrompida.")
            break # Encerra o loop interno com "break"
        classificacao = int(classificacao)
        if classificacao < 1 or classificacao > 5:
            print()
        else:
            print(f'{filme}' com {classificacao} estrelas.\n")
            break # Sai do loop interno

print()
```

Nessa solução, utilizamos “**while True:**”, que é uma técnica comum para criar loops em que a condição de parada pode

variar ou não é conhecida, como acontece nesse exemplo, visto que o usuário pode parar a classificação escolhendo 0 ou encerrá-la depois de classificar todos os cinco filmes.

Gostou dessa solução? Espero que sim! Rode esse código no [colab](#), faça modificações e “brinque” com ele. Lembre-se de que a prática é extremamente importante para alcançar melhorias.

Saiba mais

Saiba mais

1. Para entender como funciona a aplicação de processos automatizados, leia o texto [Robotic process automation e a auditoria financeira: modern framework](#), que utiliza algoritmos para automação e inteligência artificial. Note que já aprendemos sobre Python, porém existem outras linguagens semelhantes que também são adotadas para a automação. Para acessar o conteúdo sugerido, clique no link disponível a seguir.

CALÇADA, L. I. S. **Robotic process automation e a auditoria financeira: modern framework.** 2020. 52 f. Dissertação (Mestrado em Gestão de Sistemas de Informação) – Instituto Superior de Economia e Gestão, Universidade de Lisboa, Lisboa, 2020.

2. Uma leitura interessante para quem está começando a programar em Python é a do livro *[Começando a programar em Python para leigos](#)*.

MUELLER, J. P. **Começando a programar em Python para leigos.** Rio de Janeiro: Alta Books, 2020. E-book.

3. Outra dica para estudo e aprofundamento sobre esse tema é o livro *[Use a cabeça! Python](#)*.

BARRY, P. **Use a Cabeça! Python.** 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

Referências

Referências

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

CALÇADA, L. I. S. **Robotic process automation e a auditoria financeira: modern framework**. 2020. 52 f. Dissertação (Mestrado em Gestão de Sistemas de Informação) – Instituto Superior de Economia e Gestão, Universidade de Lisboa, Lisboa, 2020. Disponível em:

<https://www.repository.utl.pt/bitstream/10400.5/20874/1/D-M-LISC-2020.pdf>. Acesso em: 12 out. 2023.

GOOGLE COLAB. **Página inicial**, [s. d.]. Disponível em: <https://colab.research.google.com/>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

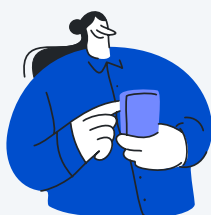
<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

Aula 4

Funções em Python

Funções em Python

Funções em Python



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

Bons estudos!

[Faça o download do arquivo](#)

Ponto de Partida

Ponto de Partida

Python é uma linguagem de programação de alto nível amplamente usada na indústria de tecnologia. Nesta aula, você começará a entender por que a

Python é tão popular e como pode ser utilizada em diversas aplicações.

Você conhecerá as ferramentas necessárias para dar início à programação em Python, o que inclui a instalação do Python em seu computador e a seleção de um ambiente de desenvolvimento adequado.

As variáveis são fundamentais na programação, pois permitem armazenar e manipular dados. Durante esta etapa de estudos, você aprenderá a criar variáveis e descobrirá os diferentes tipos de dados disponíveis em Python.

Você se lembra da nossa primeira aula, quando pedi a ajuda da Python para calcular a média e mostrar a situação dos alunos? Vamos melhorar nosso cálculo com os conhecimentos de hoje!

Vamos Começar!

Vamos Começar!

Funções built-in em Python

Desde o momento em que iniciamos nossa jornada na programação com a simples linha de código “print(“hello world”)”, já estávamos explorando funções. O que é uma função? No contexto de Python, pense em funções como ferramentas prontas para uso. Elas são como blocos de

construção pré-instalados, incorporados ao coração da linguagem Python, sem a necessidade de instalação adicional. A Python oferece uma ampla variedade de funções embutidas que podem ser utilizadas em suas tarefas de programação.

No site [python](#) constam todas as funções prontas existentes em Python. Acredito que seja muito importante você acessar essa página e conhecer as funções. Já utilizamos algumas delas em nossos estudos, como a função “**print**”, que exibe objeto no formato de texto.

Outra função que exploramos foi a “**int**”, que retorna um objeto inteiro a partir de um número ou string. Também usamos a função “**range**”, que nada mais é do que um tipo de sequência imutável.

Confira, a seguir, o exemplo que utiliza a função built-in **len()** para calcular o comprimento de uma lista e, em seguida, imprime o resultado com comentários explicativos:

```
# Cria uma lista de números
numeros = [1, 2, 3, 4, 5]

# Usa a função len() para calcular o comprimento da lista
comprimento = len(numeros)
```

```
# Imprime o comprimento da lista  
print(, comprimento)
```

Nesse caso, criamos uma lista chamada “números”, com cinco elementos. Em seguida, usamos a função `len()` para calcular o comprimento dessa lista e armazenamos o resultado na variável “comprimento”. Por fim, imprimimos o valor do comprimento com uma mensagem explicativa.

```
O comprimento da lista é: 5
```

Como já foi mencionado, é interessante que você faça mais testes e aplicações das funções, na intenção de praticar o conhecimento estudado.

Função definida pelo usuário (com retorno e parâmetro)

O repertório de 70 funções built-in (pré-construídas) da Python torna a vida do programador mais fácil. No entanto, cada problema é singular e frequentemente requer abordagens específicas. É nesse contexto que surge a necessidade de criar nossas próprias funções. Tais funções são pedaços de código que executam ações definidas por nós, os desenvolvedores. Temos o controle sobre o nome da

função, os dados que ela recebe e o resultado que produz. Isso nos permite personalizar soluções para atender às demandas específicas de nossos projetos.

Acompanhe o exemplo de uma função que calcula a soma de dois números:

```
# Definindo uma função chamada "soma"
def soma(a, b):
    resultado = a + b
    return resultado

# Chamando a função e armazenando o resultado em uma variável
resultado_soma = soma(5, 3)

# Imprimindo o resultado
print(resultado_soma)
#resultado
A soma de 5 e 3 é: 8
```

Nesse exemplo, definimos uma função chamada “soma”, que aceita dois argumentos: **a** e **b**. Dentro da função, realizamos a operação de adição entre esses dois números e retornamos o resultado. Em seguida, chamamos a função com os valores 5 e 3, e armazenamos o resultado retornado em uma variável chamada “resultado_soma”. Por fim, imprimimos o resultado.

Outro exemplo interessante de ser observado é a criação de uma função para definir se um número é par:

```
# Definindo uma função chamada "e_par"
def e_par(numero):
    if numero % 2 == 0:
        return True
    else:
        return False
```

Repare que definimos a função e utilizamos o operador "%", que, em Python, é conhecido como operador de módulo ou operador de resto. Ele é usado para calcular o resto da divisão de um número pelo outro. Em uma expressão como **a % b**, o operador % retorna o valor do resto quando **a** é dividido por **b**.

Sendo assim, se um número é dividido por 2 e tem resto zero, então esse número é par.

Vamos testar essa função para dois valores: 123.120 e 1.355.989.

```
# Testando a função
numero = 123120
if e_par(numero):
    print(f{numero} é um número par.")
```

```
else:  
    print(f{numero} não é um número par.")  
#resultado: 123120 é um número par.  
  
# Testando a função  
numero = 1355989  
if e_par(numero):  
    print(f{numero} é um número par.")  
else:  
    print(f{numero} não é um número par.")  
#resultado 1355989 não é um número par.
```

Note que a função retornou que o primeiro número é par e o segundo, não. Nesses exemplos, percebemos como é feita uma função definida por nós. Agora, faça testes e utilize sua imaginação para criar funções.

Siga em Frente...

Siga em Frente...

Funções anônimas em Python

Dentro do universo das funções em Python, existe um recurso poderoso chamado “expressão lambda”. As expressões lambda são usadas para criar funções anônimas, o que significa que elas não têm um nome definido com “def”. Tais funções são úteis quando você precisa de uma ação simples que será utilizada apenas uma vez. Para saber mais detalhes sobre esse recurso, acesse: [python](#) (Python 3.12.2 Documentation, [s. d.]b).

Confira um exemplo:

```
soma = lambda a, b: a + b
resultado = soma(3, 4)
print(resultado) # Isso imprimirá 7

#resultado: 7
```

Nesse exemplo, criamos uma expressão lambda que realiza a adição de dois números: **a** e **b**. Não atribuímos um nome à função, mas podemos usá-la como qualquer outra. Portanto, expressões lambda são úteis em situações nas quais precisamos de uma função pequena e simples para uma tarefa específica.

Até agora, conhecemos as funções prontas do Python e descobrimos como podemos desenvolver nossas próprias

funções, seja de forma mais visual (quando utilizamos def), seja de forma anônima (usando lambda).

Vamos Exercitar?

Vamos Exercitar?

Já sabemos como calcular médias. Vamos, a partir de agora, dar um upgrade no primeiro código com o qual trabalhamos utilizando o conhecimento obtido nesta aula.

```
# Lista de notas dos estudantes
notas = [7.5, 8.0, 6.5, 9.0, 7.0]

# Função regular para calcular a média
def calcular_media(notas):
    total = sum(notas)
    media = total / len(notas)
    return media

# Função lambda para arredondar a média para duas casas decimais
arredondar_media = lambda media: round(media, 2)

# Calcular a média
```

```
media = calcular_media(notas)
media_arredondada = arredondar_media(media)

# Verificar se os estudantes foram aprovados
situacao = if media_arredondada >= 7 else

# Resultados
print(, notas)
print(, media_arredondada)
print(, situacao)
```

Esse código calcula a média das notas dos estudantes, arredonda o resultado e determina se eles foram aprovados ou reprovados com base na média.

Trata-se, portanto, de um exemplo prático de uso de funções e estruturas condicionais, já que utilizamos funções prontas como “sum”, “len” e “round”, e criamos funções definidas e anônimas.

Elaboramos, na primeira aula, um código simples para calcular a média. Agora, no final desta unidade de aprendizagem, conseguimos criar nossas próprias funções e utilizar muitas ferramentas de Python!

Você se sentirá cada vez mais preparado para criar algoritmos mais robustos e aplicá-los a diversas realidades. Lembre-se de sempre praticar!

Saiba mais

Saiba mais

1. Uma leitura interessante para quem está começando a programar em Python é a do livro [*Começando a programar em Python para leigos*](#).

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book.

2. Outra dica para estudo e aprofundamento sobre esse tema é o livro [*Use a cabeça! Python*](#).

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book.

3. Para saber mais detalhes sobre a aplicação da linguagem Python, sugiro a leitura do texto [*Normalização de dados textuais com Python*](#).

TURKEL, J.; CRYMBLE, A. Normalização de dados textuais com Python. **The Programming Historian**, 17 jul. 2012.

Referências

Referências

6. EXPRESSIONS. **Python 3.12.2 Documentation**, [s. d.]b.

Disponível em:

<https://docs.python.org/3/reference/expressions.html#lambda>. Acesso em: 12 out. 2023.

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

BUILT-IN Functions. **Python 3.12.2 Documentation**, [s. d.]a.

Disponível em:

<https://docs.python.org/3/library/functions.html>. Acesso em: 12 out. 2023.

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em:

<https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.

TURKEL, J.; CRYMBLE, A. Normalização de dados textuais com Python. **The Programming Historian**, 17 jul. 2012. Disponível em:

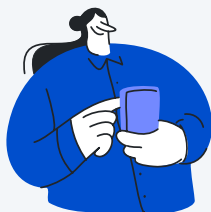
<https://programminghistorian.org/pt/licoes/normalizacao-dados-textuais-python>. Acesso em: 12 out. 2023.

Aula 5

Introdução a Linguagem Python

Videoaula de Encerramento

Videoaula de Encerramento



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Estudante, esta videoaula foi preparada especialmente para você. Nela, você irá aprender conteúdos importantes para a sua formação profissional. Vamos assisti-la?

Bons estudos!

[Faça o download do arquivo](#)

Ponto de Chegada

Ponto de Chegada

Olá, estudante! Para desenvolver a competência associada a esta unidade de aprendizagem, que é “Desenvolver o pensamento lógico para a resolução de problemas a partir da programação utilizando a linguagem Python”, devemos, antes de tudo, conhecer os conceitos fundamentais da linguagem Python.

Isso inclui compreender sua sintaxe, as variáveis, os tipos de dados e as estruturas de controle. Tais conceitos servirão como alicerce para o aprimoramento de sua capacidade de resolver problemas por meio da programação (Manzano; Oliveira, 2019).

Ao longo desta etapa de estudos, foi possível relacionar esses conceitos básicos com a construção de scripts. Você também aprendeu a utilizar estruturas condicionais para tomar decisões em seus programas, o que é essencial para criar soluções flexíveis. Além disso, exploramos as estruturas de repetição para automatizar tarefas repetitivas e criar programas mais eficientes.

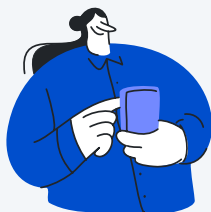
Uma parte crucial para o desenvolvimento do pensamento lógico é a criação de funções em Python (Mueller, 2020). As funções são blocos de código reutilizáveis que permitem dividir problemas complexos em partes menores, tornando-os mais fáceis de se gerenciar e resolver.

Você não apenas aprendeu sobre os conceitos e técnicas relacionados a esse contexto, mas também se tornou capaz de aplicar esse conhecimento em situações práticas. Ao construir scripts e funções em Python, você desenvolveu a habilidade de analisar problemas, dividindo-os em etapas lógicas e elaborando soluções algorítmicas (Barry, 2018).

A competência desta unidade de aprendizagem permite que você se torne um solucionador de problemas com habilidades em Python, preparando-o para enfrentar desafios tecnológicos e computacionais.

É Hora de Praticar!

É Hora de Praticar!



Este conteúdo é um vídeo!

Para assistir este conteúdo é necessário que você acesse o AVA pelo computador ou pelo aplicativo. Você pode baixar os vídeos direto no aplicativo para assistir mesmo sem conexão à internet.

Para contextualizar sua aprendizagem, imagine a seguinte situação: você trabalha em uma loja de eletrodomésticos e precisa criar uma calculadora de desconto em Python para ajudar os vendedores a calcularem o valor final de uma compra com base no preço do produto e em um desconto percentual oferecido.

Questões norteadoras:

1. Como você pode aplicar seus conhecimentos em programação em Python para criar uma calculadora de desconto?
2. Que estruturas condicionais em Python você pode usar para verificar se o desconto está dentro de limites aceitáveis?

Refleta

Para encerrar e consolidar seu aprendizado, reflita sobre as seguintes perguntas:

1. Como as estruturas condicionais em Python podem ser usadas para tomar decisões em programas?
2. Qual é a importância da reutilização de código por meio de funções na programação em Python?
3. Como você pode aplicar o pensamento lógico para resolver problemas complexos usando Python em sua trajetória acadêmica e profissional?

Esses questionamentos ajudarão você a internalizar ainda mais os conhecimentos adquiridos e a perceber a amplitude de sua aplicação. Desejo sucesso em sua jornada de aprendizagem!

Resolução do Estudo de Caso

Vamos resolver o desafio seguindo um passo a passo.

Nesse estudo de caso, desenvolveremos um programa em Python para calcular o valor final de uma compra com desconto. A principal competência é a aplicação do

pensamento lógico para construir um programa funcional, que ajude os vendedores a calcularem o preço final.

Confira, a seguir, o código Python para criar a calculadora de desconto:

```
# Solicita ao usuário que insira o valor do produto e o percentual de
desconto
valor_produto = float(input())
percentual_desconto = float(input())

# Verifica se o percentual de desconto está dentro dos limites
aceitáveis (0-100%)
if percentual_desconto < 0 or percentual_desconto > 100:
    print()
else:
    # Calcula o valor do desconto
    desconto = valor_produto * (percentual_desconto / 100)

    # Calcula o valor final da compra
    valor_final = valor_produto - desconto

    # Exibe o valor final da compra
    print(f{valor_final:.2f})
```

Um exemplo de resultado:

```
Digite o valor do produto: R$ 150
Digite o percentual de desconto: 12.5
```

Valor com desconto: R\$ 131.25

Nesse código, o programa solicita ao usuário que insira o valor do produto e o percentual de desconto. Em seguida, ele verifica se o percentual de desconto está dentro dos limites aceitáveis (entre 0% e 100%). Se estiver em conformidade, o programa calcula o valor do desconto e o valor final da compra, exibindo o resultado na tela.

Assimile

O material visual a seguir esquematiza os principais tópicos abordados nesta unidade de aprendizagem, em que apresentamos uma introdução à linguagem Python. Este infográfico exibe uma percepção clara e sucinta de cada parte dessa etapa de estudos, enfatizando os conceitos e fundamentos necessários para uma boa compreensão dos saberes desenvolvidos.



Figura 1 | Infográfico: introdução à linguagem Python. Fonte: elaborada pelo autor.

Referências

BARRY, P. **Use a Cabeça! Python**. 2. ed. Rio de Janeiro: Alta Books, 2018. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555207842>. Acesso em: 12 out. 2023.

MANZANO, J. A. N. G.; OLIVEIRA, J. F. de. **Algoritmos: lógica para desenvolvimento de programação de computadores**. 29. ed. São Paulo: Érica, 2019.[Fd1]

MUELLER, J. P. **Começando a programar em Python para leigos**. Rio de Janeiro: Alta Books, 2020. E-book. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9786555202298>. Acesso em: 12 out. 2023.